

CI/CD Automatic Deployment Documentation

GitHub Actions, Docker Hub, and AWS EC2 Automation Guide

1. Objective

The primary goal of this implementation is to design and deploy a completely automated Continuous Integration and Continuous Deployment (CI/CD) pipeline. By eliminating manual deployment steps, every code modification pushed to the primary repository branch is automatically built, verified, and updated on the live application server, guaranteeing fast, reliable, and hands-free software delivery.

2. System Architecture

The automated software pipeline executes sequentially through the following decoupled components to deliver code updates from the developer's workstation to the final production platform:

Developer → GitHub Repository → GitHub Actions → Docker Hub → AWS EC2 → Live Website

3. Tools & Technologies

Tool / Technology	Purpose in Pipeline
Git & GitHub	Version control system and remote source code repository management.
GitHub Actions	Automation engine utilized to trigger workflows, build Docker images, and manage remote execution.
Docker & Compose	Containerization framework to bundle applications consistently across staging and live server environments.
Docker Hub	Secure container registry used for hosting, versioning, and distributing production-ready images.
AWS EC2	Cloud infrastructure provider running Ubuntu 24.04 LTS to serve the application live over the web.
SSH Protocol	Secure communication mechanism enabling automated pipeline execution on the remote live host.

4. Repository & Application Setup

Execute the following baseline environment setups locally to initialize the source control tracking configuration and populate initial application mock assets:

```
# Clone the remote tracking repository
git clone https://github.com/rejin-cloud/auto-deploy-app.git
cd auto-deploy-app

# Construct workspace directories and mock data
mkdir app
echo "<h1>Hello CI/CD</h1>" > app/index.html

# Snapshot changes and update remote branch
git add .
git commit -m "Initial commit"
git push origin main
```

4.1 Dockerfile Configuration

Create a standalone `Dockerfile` at the root level of your project directory to dictate how the application environment should containerize:

```
FROM nginx:latest
COPY app/ /usr/share/nginx/html/
EXPOSE 80
```

4.2 Local Docker Compose Specification

Incorporate a `docker-compose.yml` mapping file into the repository root directory to standardise localized multi-container service setups:

```
services:
  web:
    image: rejin1421/auto-deploy-app:latest
    container_name: auto-deploy-app
    ports:
      - "80:80"
    restart: always
```

5. Remote Server Configuration (AWS EC2)

Log in via secure shell credentials to your remote AWS EC2 instance executing Ubuntu 24.04 LTS, and execute the initialization steps outlined below.

5.1 Docker Engine Installation

Update local package indices and provision the core virtualization layer dependencies on the remote node:

```
apt update
apt install docker.io -y
systemctl start docker
systemctl enable docker
```

5.2 Docker Compose Plugin Provisioning

Configure and fetch the modular binary runtime architecture required to parse advanced Compose execution scripts directly:

```
mkdir -p ~/.docker/cli-plugins/
curl -SL https://github.com/docker/compose/releases/latest/download/docker-compose-linux-x86_64 -o ~/.docker/cli-plugins/docker-compose
chmod +x ~/.docker/cli-plugins/docker-compose

# Verify proper operational response
docker compose version
```

5.3 Deployment Space Initialization

Set up specific dedicated host paths configured explicitly to anchor remote runtime execution artifacts:

```
mkdir -p /opt/auto-deploy-app
cd /opt/auto-deploy-app
```

Construct or upload a duplicated `docker-compose.yml` within `/opt/auto-deploy-app/` to maintain consistent live orchestration rules:

```
services:
  web:
    image: rejin1421/auto-deploy-app:latest
    container_name: auto-deploy-app
    ports:
      - "80:80"
    restart: always
```

5.4 Pipeline SSH Authentication Configuration

Provision strict access keys on the instance to guarantee GitHub actions workflows possess adequate permission sets without manual interactions:

```
ssh-keygen -t rsa -b 4096
cat ~/.ssh/github_actions_deploy.pub >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
```

6. Pipeline Secrets Configuration

To ensure credential containment, define the following explicit variables within your secure project settings console under **GitHub Settings** → **Secrets and Variables** → **Actions**:

- **DOCKER_USERNAME**: Authorized username identifier associated with the target Docker Hub registry account.
- **DOCKER_PASSWORD**: Secret password token associated with your Docker Hub integration.
- **SERVER_IP**: The publicly routing IPv4 address interface mapping to your targeted AWS EC2 node.
- **SERVER_USER**: Administrative remote context access username identifier (e.g., **ubuntu**).
- **SERVER_SSH_KEY**: The private authentication key credential generated previously to permit unattended actions pipeline login.

7. GitHub Actions Pipeline Definition

Populate a workflow configurations document named exactly **.github/workflows/deploy.yml** inside your project workspace root layout:

```

name: CI/CD Auto Deployment

on:
  push:
    branches:
      - main

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Source Code Checkout
        uses: actions/checkout@v4

      - name: Authenticate Registry Access
        uses: docker/login-action@v3
        with:
          username: ${ secrets.DOCKER_USERNAME }
          password: ${ secrets.DOCKER_PASSWORD }

      - name: Build Container Image
        run: docker build -t ${ secrets.DOCKER_USERNAME }/auto-deploy-app:latest .

      - name: Distribute Container Image
        run: docker push ${ secrets.DOCKER_USERNAME }/auto-deploy-app:latest

      - name: Remote Deployment Executions
        uses: appleboy/ssh-action@v1.0.3
        with:
          host: ${ secrets.SERVER_IP }
          username: ${ secrets.SERVER_USER }
          key: ${ secrets.SERVER_SSH_KEY }
          script: |
            cd /opt/auto-deploy-app
            sudo docker compose pull
            sudo docker compose up -d

```

8. Pipeline Verification & Operational Testing

To evaluate whether the orchestrated deployment flows function seamlessly, execute a local file change lifecycle verification run:

```
# Apply updates to the entrypoint interface
echo "<h1>Hello CI/CD - Automated Update Verified</h1>" > app/index.html

# Broadcast changes to trigger automation
git add .
git commit -m "Test automatic deployment"
git push origin main
```

Monitor execution statuses via the GitHub UI panel. Once successfully completed, access your destination browser endpoint to confirm modifications have synchronized properly on the hosting instance.

9. Rollback Procedures

If critical system regressions occur or unexpected bugs manifest, restore immediate cluster integrity using standard reversion strategies:

```
# Revert the faulty commit tracking state
git revert <commit_id>

# Re-submit structural changes to trigger rollback build
git push origin main
```

System Outcome Status

The automated workflow successfully delivers a secure, containerized software lifecycle. Code check-ins directly initiate cloud builds, transfer valid updates to central Docker registries, and configure running hosts with no manual human reliance.